

SSU (ABNT NBR 14522) na TON-V2 — guia de referência

v1.3 · 21/09/2026 · protocolo, firmware, payload, testes e bancada · Aupus Energia · Engenharia IoT · **revisão v1.3:** nomes dos registradores reativos alinhados ao A-966 real; RX-timeout, wrap de 15 bits e reinício de intervalo aplicados no código; caminho LoRa binário.

Resumo

O que chega	1 bloco por segundo: segundos restantes do intervalo, posto, quadrante e 2 contadores de pulsos (ativa, reativa) acumulados desde o início do intervalo.
Energia	Δ pulsos do mesmo registrador × Ke (kWh/pulso). Nunca comparar contadores de registradores diferentes.
Enlace	110 baud, 8N1, coletor aberto, sem delimitador — o bloco é fechado pelo silêncio.
Hardware	Borne X14 (X14-2 = SU+, X14-1 = GND) → IO48 · pull-up R47 680 Ω · TVS D4 · UART0 (Serial0), só RX.
Saída	Bucket por intervalo de demanda (15 min), JSON no formato do A-966, categoria Gateway no NexON. Em satélite LoRa: JSON plano → frame binário LORA_TYPE_SSU.

Camada física

	Normal (3.4.1)	Estendido (3.4.2)
Velocidade	110 baud ±3 %	110 baud ±3 %
Caractere	start + 8 + 1-2 stop	start + 8 + 1 stop
Bloco / transmissão	8 octetos / ≈ 800 ms	9 octetos / ≈ 818 ms
Silêncio mínimo	≈ 200 ms	≈ 182 ms

Bit = 9,09 ms; caractere ≈ 91 ms. Receber sempre em SERIAL_8N1. RX não invertido (idle alto pelo pull-up). Fechar o bloco em qualquer silêncio entre ~10 ms e 182 ms.



RX-timeout do ESP32 é em **símbolos** (caracteres), não bits. A 110 baud 1 símbolo ≈ 91 ms. Usar setRxTimeout(1); com 4 (≈ 364 ms) o bloco nunca fecha — corrigido no gerador em 21/09.

Formatos de bloco

Modo fixo de fábrica. O octeto 3 muda de significado entre os formatos → autodetecção: aceitar o formato só após **10 blocos seguidos** com tamanho + checksum válidos. Blocos da detecção não acumulam.

Normal — 8 octetos

1	2	3	4	5	6	7	8
seg[7:0]	seg[11:8] + flags	segmento + tarifa	ativa lo	ativa hi · 7 bits	reativa lo	reativa hi · 7 bits	LRC

Octeto	Bits	Conteúdo
1-2	1[7:0], 2[3:0]	Segundos restantes (12 bits, regressivo)
2	6-7	Pulsos capacitivos/indutivos computados para UFER/DMCR
3	0-3 · 4-5	Segmento horário (2 = fora de ponta nos vetores) · tarifa: 0 azul, 1 verde, 2 irrigantes, 3 outras
4-7	15 bits LE	Contadores ativa / reativa — máscara 0x7F no octeto alto; wrap em 32 768
8	—	LRC = ~(XOR dos octetos 1-7)

Estendido — 9 octetos (obrigatório com geração)

1	2	3	4	5	6	7	8	9
seg[7:0]	seg[11:8] + bits 4/5/6-7	posto + quadrante	ativa lo	ativa hi	reativa lo	reativa hi	CRC lo	CRC hi

Octeto	Bits	Conteúdo
1-2	1[7:0], 2[3:0]	Segundos restantes (12 bits, regressivo)
2	4	Toggle a cada reposição de demanda (detectar mudança)
2	5	Toggle a cada fim de intervalo reativo (detectar mudança)
2	6-7	Posto reativo: 0 nenhum, 1 capacitivo, 2 indutivo, 3 ambos
3	0-1	Posto horário: 1 ponta, 2 fora de ponta, 3 quarto posto
3	2-3	Reservados — mascarar
3	4-5	Quadrante por tabela: 00→Q1, 01→Q4, 10→Q2, 11→Q3
3	7	Reativo sendo tarifado agora
4-7	16 bits LE	Contadores ativa / reativa: lo (hi << 8); wrap em 65 536
8-9	—	CRC-16/ARC: polinômio refletido 0xA001, init 0x0000, lo primeiro

Validação e decodificação

Checksum inválido → descartar o bloco inteiro e contar em ssu_err.

```

uint8_t lrc(const uint8_t* d, size_t n) {
    uint8_t v = 0;
    for (size_t i = 0; i < n; i++) v ^= d[i];
    return (uint8_t)~v;
}
uint16_t crc16(const uint8_t* d, size_t n) {
    uint16_t c = 0; // ≠ Modbus (0xFFFF)
    for (size_t i = 0; i < n; i++) {
        c ^= d[i];
        for (int b = 0; b < 8; b++)
            c = (c & 1) ? (c >> 1) ^ 0xA001 : c >> 1;
    }
    return c;
}

```

```

seg      = d[0] | ((d[1] & 0x0F) << 8);
bitRepos = d[1] & 0x10;      // toggle
bitFimR  = d[1] & 0x20;      // toggle
postoR   = (d[1] >> 6) & 3;   // 0 · cap · ind · ambos
postoH   = d[2] & 3;         // 1 ponta · 2 fora · 3 quarto
q        = QUAD[(d[2] >> 4) & 3]; // {1,4,2,3}
tarifR   = d[2] & 0x80;
ativa    = d[3] | (d[4] << 8);
reativa  = d[5] | (d[6] << 8);
// normal: ativa = d[3] | ((d[4] & 0x7F) << 8)
//          seg.  = d[2] & 0x0F; tarifa = (d[2]>>4)&3

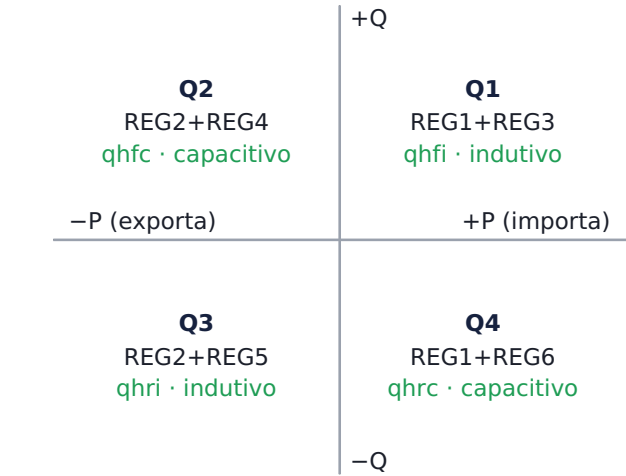
```

Exemplo: 3E 80 81 C0 2B 8E 12 55 B8

Octetos	Cálculo	Resultado
3E 80	0x3E + (0x80 & 0x0F)·256	62 s
80	b7-6 = 10	posto reativo indutivo
81	b1-0 = 01 · b5-4 = 00 · b7 = 1	ponta · Q1 · tarifando reativo
C0 2B / 8E 12	0x2BC0 / 0x128E	11 200 (REG1) / 4 750 (REG3)
55 B8	CRC = 0xB855	válido

Quadrantes e registradores

Q	bits	Registradores	JSON	Sentido
Q1	00	REG1 + REG3	phf · qhfi	importa · indutivo
Q2	10	REG2 + REG4	phr · qhfc	exporta · capacitivo
Q3	11	REG2 + REG5	phr · qhri	exporta · indutivo
Q4	01	REG1 + REG6	phf · qhrc	importa · capacitivo



Reg	Conteúdo (NBR)	JSON (A-966)
REG1	ativa direta	phf
REG2	ativa reversa	phr
REG3	reativa Q1	qhfi
REG4	reativa Q2	qhfc
REG5	reativa Q3	qhri
REG6	reativa Q4	qhrc

```
regAtiva    = (q == 1 || q == 4) ? 1 : 2;
REG_R[5]    = { 0, 3, 4, 5, 6 };
regReativa  = REG_R[q];
// JSON: acum[3]→qhfi acum[4]→qhfc
//       acum[5]→qhri acum[6]→qhrc
```

Convenção dos nomes (a do A-966, que o NexON consome): **f/r** = sinal de Q (fornecida = +Q, recebida = −Q); **i/c** = Q1 e Q3 indutivo, Q2 e Q4 capacitivo — sempre relativo ao sentido do fluxo ativo (o mesmo critério da ANEEL para geração). O backend soma qhfi+qhri como indutivo e qhfc+qhrc como capacitivo.

Base da correção (v1.3): 400 buckets reais do "Gateway IoT A-966 1" (UFV, 17/09/2026) cruzados com o quadrante do frame: ativa Q2/Q3 → phr, Q4 → phf (tabela {1,4,2,3} confirmada); reativa **Q2 → qhfc** (57×), **Q3 → qhri** (132×), **Q4 → qhrc** (158×). O v1.2 trazia Q2→qhri / Q3→qhrc / Q4→qhfc — invertido: uma UFV exportando em Q2 apareceria como indutiva.

Não calcular quadrante como "bits + 1". Acerta só Q1: 01 vira Q2 (consumo contado como injeção), 10 vira Q3 (reativo trocado), 11 vira Q4 (injeção contada como consumo). Sem erro visível — o checksum é válido.

Regras de acumulação

Situação	Sintoma	Tratamento
Troca de quadrante	Contador da ativa cai (11 210 → 41)	É outro registrador. Δ contra o último valor do REG2 — 0 se o intervalo foi visto desde o início (reinício observado ⇒ todos os registradores partiram de 0). Baseline só se o REG2 nunca foi visto antes de um reinício (boot no meio do intervalo).
Wrap	65 500 → 40 no mesmo registrador	Δ modular: 76. Normal (15 bits): mod 32 768 — 32 700 → 40 = 108.
Reinício do intervalo	Segundos sobem (0 → 899)	Publicar bucket, zerar acumuladores, base de todos = 0 e todos "vistos"; o 1º bloco novo já conta.
Fechamento 3×	Blocos idênticos repetidos	Δ já dá 0; contar o evento de fechamento uma vez só.
Bits 4 e 5	Invertem a cada evento	Detectar mudança contra o último valor. Bit 5 ≈ a cada 4 intervalos (reativo 1 h).

```
const uint16_t MASK = normal ? 0x7FFF : 0xFFFF;
void processa(uint8_t reg, uint16_t atual) {      // reg = 1..6
    if (visto[reg]) acumula(reg, (atual - ultimo[reg]) & MASK);
    ultimo[reg] = atual; visto[reg] = true;
}
void reinicioIntervalo() {
    publicaBucket(); zeraAcumuladores();
    for (int r = 1; r <= 6; r++) { ultimo[r] = 0; visto[r] = true; }
}
```

Firmware na TON-V2

```
static HardwareSerial& ssu = Serial0; // UART0 = o objeto do core (console é USB-CDC); nunca 2ª instância
ssu.begin(110, SERIAL_8N1, 48, -1); // RX = I048, sem TX
ssu.setRxTimeout(1); // 1 símbolo ≈ 91 ms — dispara no silêncio de 182 ms
ssu.onReceive(onBloco, true); // callback só copia para fila FreeRTOS
```

Item	Comportamento
Parser	Roda no loop, lendo da fila. A recepção não depende do loop (SD, W5500, LoRa e Modbus bloqueiam por centenas de ms).
Bucket	acum[REG1..REG6]; publicado quando os segundos reiniciam — alinhado ao intervalo faturado.
NVS	Acumuladores e NSU salvos a cada publicação e a cada 60 s.
Enlace degradado	30 blocos inválidos seguidos → sts = 0 e volta à autot detecção.
Alarme de formato	Bloco normal em instalação com geração → pedir saída estendida à concessionária.
Diagnóstico	/diagnostics: ssu_ok, ssu_err, ssu_fmt, ssu_degradado, ssu_intervalos.
Gerador	Com "Medidor Concessionária (SSU)" no diagrama, o IO48 vira RX (sai do /inputs) e é gerado ssu.cpp. Lib: AupusNex0n/firmware-libs/ssu_nbr14522/.
LoRa (TON4v2 satélite)	O JSON aninhado do A-966 tem ~236 B e não cabe no MTU de 200 B do E220. No satélite (SSU_LORA_BIN=1) o bucket sai em JSON plano (sem frame/ver/time) e vira o frame binário LORA_TYPE_SSU (0x11, ≈ 90 B no ar); o gateway carimba a hora e republica plano; o backend lê phf/phr/qh* tanto em data. {} quanto no topo. Gateway precisa de firmware com a tabela nova (regravar).

Payload e contas no NexON

```
{ "NSU": 32, "ver": "2.0.0-...",
  "data": { "cdo": "15", "phf": 0, "phr": 1532, "sts": 1, "qhfi": 0, "qhfc": 4, "qhri": 0, "qhrc": 0,
    "frame": "00 90 a2 fc 05 04 00 c6 18" },
  "time": "1789661700", "ke": 0.048, "fmt": 2, "q": 2, "seg": 0, "posto": 2, "ssu_err": 0 }
```

Campo	Significado
phf ... qhrc	Pulsos crus do bucket, 1:1 com REG1...REG6 (nomes da tabela acima)
ke	kWh/pulso do cadastro do medidor; sem ele o backend usa 0,048
fmt · q · seg · posto	Formato (1 normal, 2 estendido), quadrante, segundos e posto do último bloco
sts · ssu_err	Estado do enlace · blocos inválidos

Contas (bucket de 15 min): kWh = pulsos × Ke; kW médio = kWh ÷ 0,25. Ex.: 1 532 × 0,048 = 73,5 kWh → 294 kW exportados.

Exemplo real do A-966 = caso de teste 9. O frame acima decodifica como Q2 (octeto 3 = A2 → bits 5-4 = 10), ativa 1 532 no REG2 e reativa 4 no REG4 → phr = 1532, qhfc = 4 — exatamente o que o A-966 publicou. O ssu.cpp gerado emite o mesmo.

Testes automatizados

Vetor	Esperado
1C 82 92 D0 20 1C 0C 13	normal · 540 s · fora de ponta · verde · 8 400 / 3 100 · LRC 0x13
3E 80 81 C0 2B 8E 12 55 B8	Q1 · 62 s · ponta · indutivo · 11 200 (REG1) / 4 750 (REG3) · CRC 0xB855
82 43 32 29 00 0C 00 98 63	Q3 · 898 s · fora de ponta · capacitivo · 41 (REG2) / 12 (REG5) · CRC 0x6398
01 00 12 16 00 0a 00 f0	normal · 1 s · fora de ponta · verde · 22 / 10 · LRC 0xF0
00 90 a2 fc 05 04 00 c6 18	Q2 · 0 s · fora de ponta · indutivo (posto reativo) · 1 532 (REG2) / 4 (REG4) · CRC 0x18C6

#	Caso (test_ssu.cpp)	Critério	Status
1	Bloco normal no parser estendido	Rejeitado	ok
2	Q1 → Q3 sem reinício visto (boot no meio)	Sem pico; REG2 = baseline	ok
2b	Q1 → Q3 com reinício visto	Δ REG2 = 41, Δ REG5 = 12 (não baseline)	ok novo
3	Q1 → Q4	Ativa continua no REG1	ok
4	3 fechamentos idênticos	1 evento	ok
5	Bits 4/5 alternando	1 evento por transição	ok
6	CRC/LRC com 1 bit errado	Rejeitado	ok
7	Wrap 16 bits 65 500 → 40	Δ = 76	ok
8	Wrap 15 bits 32 700 → 40 (normal)	Δ = 108	ok novo
9	Frame Q2 do payload (smoke do gerador)	phr = 1532, qhfc = 4 — ordem qhfi,qhfc,qhri,qhrc ← REG3..6	ok
10	Enquadramento a 110 baud (simulador)	Todo callback com 8 ou 9 bytes, 1/s	bancada

Bancada: E750 + Conprove CE-6003

Ligação (tutorial Conprove E750 A2E3 — conferir na tampa do medidor)

Mala	E750	Obs.
V1, V2, V3 / comum	1, 2, 3 / 5	Medidor é autoalimentado: sem tensão, sem SSU
I1, I2, I3 → retornos	8, 9, 10 → 15, 14, 13	Retornos em ordem invertida
—	SU+ / SU−	X14-2 / X14-1 da TON-V2 (direto, sem resistor em série)

Ângulos (tensões fixas em 0°, −120°, +120°; $\varphi = \angle V - \angle I$)

Alvo	φ	I1	I2	I3	SSU esperada	JSON
Q1	+25°	−25°	−145°	+95°	REG1 + REG3	phf · qhfi
Q4	−25°	+25°	−95°	+145°	REG1 + REG6	phf · qhrc
Q2	+155°	−155°	+85°	−35°	REG2 + REG4	phr · qhfc
Q3	−155°	+155°	+35°	−85°	REG2 + REG5	phr · qhri

FP 0,92 = ±23,07°. No 1º ponto, conferir o quadrante no mostrador do E750; se vier invertido, trocar o sinal de φ em tudo.

Roteiro

- Só tensão: SSU transmitindo, formato travado, segundos decrescendo, contadores parados.
- Ke de bancada: Q1, $\varphi = 0$, 15 min (ex.: 66,4 V × 5 A × 3 ≈ 996 W → 0,249 kWh). Ke = kWh injetados ÷ pulsos. Repetir com $\varphi = 90^\circ$ para o reativo.
- Um quadrante por intervalo: Q1, Q4, Q2, Q3 — energia só nos 2 registradores do quadrante; conferir que o JSON cai em qhfi / qhrc / qhfc / qhri, nessa ordem.
- Transições (Sequencer): Q1→Q4, Q1→Q3, Q4→Q3 no meio do intervalo, várias vezes — sem pico.
- FP: $\varphi = 23,0^\circ$ e 23,2° — FP do NexON = cos φ injetado.
- Longa duração: cronometrar o bit 5, observar ssu_err, reboot no meio do intervalo (NVS).

Pendências

#	O quê	Onde	Critério de fechamento	Status
1	Valor de setRxTimeout no código gerado	_genSsuCpp	Teste 10	aplicado 21/09 (1 símbolo) · confirmar na bancada
2	Caso Q1→Q3 do test_ssu.cpp	lib	Δ REG2 = 41 com intervalo visto	aplicado (caso 2b)
3	Wrap de 15 bits no bloco normal	lib	Teste 8	aplicado
4	Mapeamento de registrador no payload	ssu.cpp	Teste 9 — qhfc = REG4 (A-966)	aplicado
5	Uma só instância de UART0	código gerado	Serial0 por referência, sem HardwareSerial(0)	aplicado
6	Spec: Serial2, "bits+1 erra 2 de 4", ângulos 155/205, nomes reativos	spec	Spec alinhada a este guia	aberto
7	Convenção de ângulo do CTC; faixas da CE-6003	mala / CTC	Quadrante no mostrador = tabela	aberto
8	Ke de cadastro primário × Ke de bancada	parametrização E750	$Ke_{prim} = Ke_{sec} \times RTC \times RTP$, se aplicável	aberto
9	Contador 899→0 ou 900→1; intervalo reativo	bancada	Captura de uma virada; bit 5 cronometrado	aberto
10	Bucket em satélite LoRa	gerador (LORA_TYPE_SSU)	Bucket do satélite chega no NexON via gateway gravado	aplicado · bancada

Referências

ABNT NBR 14522:2008	§3.4.1 normal, §3.4.2 estendido, §3.4.2.6 registradores
Landis+Gyr E750 (A2E3)	Modo e constante da SU, intervalos de demanda e reativo
ESP32-S3 TRM / HardwareSerial.h	RX timeout em símbolos, onReceive, GPIO matrix
Conprove	Manual da CE-6003, CTC (Quick, Sequencer), tutorial E750 A2E3
Internos	Spec "Firmware de leitura da SSU" · IOT-SSU-NBR14522-TON-V2-INTEGRACA0.md · REVISA0-SSU-GUIA-v1.2-HW-v1.1.md · firmware-libs/ssu_nbr14522/ · mqtt.service.ts > salvarDadosGateway · guia de hardware da TON-V2 (v1.2)

Histórico: v1.2 (21/09) revisão inicial · v1.3 (21/09) nomes reativos pela convenção do A-966 (evidência: 400 buckets reais), exemplo do A-966 promovido a referência/caso 9, LoRa via frame binário, RX-timeout 1 símbolo, wrap 15 bits e "visto" no reinício aplicados na lib (55/55), pendências 1-5 fechadas.